REQUISITOS NO FUNCIONALES

1. Disponibilidad de la plataforma web: El portal web comercial debe garantizar un uptime mínimo del 99.9%. Significa que la página para los clientes debe estar operativa y accesible casi la totalidad del tiempo, minimizando caídas del servicio.
2. Tolerancia a la concurrencia: La base de datos y la web deben soportar múltiples conexiones asincrónicas simultáneas. Significa que muchos usuarios pueden buscar y comprar boletos al mismo tiempo sin colapsar el servidor.
3. Consumo de memoria RAM optimizado: La aplicación Java Desktop debe mantener un uso eficiente de los recursos de hardware locales. Significa que el programa no debe ralentizar las computadoras operativas de los aeropuertos.
4. Transacciones ACID: El motor SQL debe asegurar atomicidad, consistencia, aislamiento y durabilidad. Significa que los datos financieros y de asientos se graban por completo o no se graban, evitando registros corruptos.
5. Seguridad frente a inyecciones de código: Las consultas DAO deben utilizar obligatoriamente sentencias preparadas (PreparedStatement). Significa que el sistema bloquea ataques maliciosos que intenten manipular la base de datos a través de campos de texto.
6. Manejo local de excepciones: El backend de Java debe capturar fallas mediante bloques try-catch estructurados. Significa que si ocurre un error con la base de datos, el programa no se cierra abruptamente, sino que avisa al usuario.
7. Baja latencia en consultas secundarias: El submódulo de inteligencia competitiva por web scraping debe operar bajo rangos de tiempo estrictos. Significa que la recolección externa de precios no debe demorar los procesos internos de la empresa.
8. Integridad referencial estricta: Las relaciones de la base de datos deben validarse mediante claves primarias y foráneas rigurosas. Significa que el motor SQL impide cargar un boleto o pago asociado a un pasajero o vuelo que no existan.
9. Cierre automatizado de recursos: El sistema de consultas de búsqueda debe implementar bloques try-with-resources. Significa que las conexiones a la base de datos se liberan solas al finalizar, previniendo fugas de memoria.
10. Desacoplamiento de interfaces (Frontend/Backend): El portal web comercial debe estar completamente separado de la aplicación de escritorio. Significa que se pueden modificar los diseños visuales de la web sin alterar el código de gestión interno.
11. Uso de un Look and Feel unificado: Las pantallas Java deben inicializar la librería externa FlatLaf. Significa que todas las ventanas del operador tienen una estética moderna, limpia y uniforme en escala de grises y azules.
12. Nomenclatura estandarizada en código: El desarrollo en Java debe aplicar obligatoriamente el estilo de escritura CamelCase. Significa que el código fuente es prolijo, legible y fácil de mantener por cualquier desarrollador del equipo.
13. Estándar en base de datos: Los scripts SQL deben unificar el uso de mayúsculas sostenidas para palabras clave y tipos de datos. Significa que las sentencias se escriben bajo normas formales (VARCHAR, INT) para mantener la prolijidad técnica.
14. Liberación eficiente de memoria volátil: Los flujos de navegación gráfica deben destruir los marcos previos con el método dispose(). Significa que al cambiar de pantalla no se acumulan ventanas invisibles que sobrecarguen la computadora.
15. Alta cohesión y bajo acoplamiento: Las clases comerciales deben resolver sus responsabilidades de manera nativa en sus constructores. Significa que los cambios en la lógica financiera no rompen o afectan directamente a las pantallas visuales.

16. Nomenclatura uniforme de parámetros SQL: Todos los parámetros de los procedimientos almacenados deben usar el prefijo p_. Significa que los desarrolladores pueden distinguir fácilmente las variables internas de las columnas reales de las tablas.
17. Aislamiento de la persistencia: La lógica SQL embebida en Java debe ser reemplazada por llamadas estructuradas a procedimientos almacenados. Significa que el programa Java solo pide datos y delega la ejecución de las consultas pesadas al servidor MySQL.
18. Restricciones de dominio rígidas: Los campos de estado de pago o check-in deben limitarse mediante tipos de datos ENUM. Significa que la base de datos rechaza cualquier texto que no coincida exactamente con las opciones permitidas.
19. Seguridad activa en cascada: Las bajas de usuarios en el histórico deben configurarse bajo la propiedad ON DELETE SET NULL. Significa que si se borra una cuenta de acceso, los registros contables y de auditoría no se eliminan ni rompen el sistema.
20. Tolerancia a fallos en el transporte de datos de red: Los sectores críticos de la empresa deben contar con topologías físicas en malla. Significa que si un router o cable falla en operaciones o despacho, la red sigue funcionando por caminos alternativos.
21. Jerarquización de la infraestructura de red: Las subredes operativas deben estructurarse mediante switches multicapa y routers distribuidos. Significa que el tráfico de datos del aeropuerto se ordena por sectores para evitar cuellos de botella.
22. Redirección eficiente de paquetes broadcast: Los enrutadores deben incorporar directivas ip helper-address. Significa que las solicitudes de red dinámicas (DHCP) cruzan los diferentes sectores sin ser bloqueadas por los routers.
23. Escalabilidad de la infraestructura de red: Los routers Cisco utilizados deben admitir la adición de módulos de expansión físicos (NIM-2T). Significa que la red de la aerolínea puede agrandarse e interconectar nuevas sedes sin cambiar los equipos principales.
24. Organización semántica del repositorio: El control de versiones en GitHub debe aplicar de forma estricta la convención AA.B.C. Significa que el historial de archivos se ordena de forma cronológica exacta para facilitar las auditorías.
25. Documentación técnica descentralizada: Cada rama (branch) de asignaturas en GitHub debe contar con su propio archivo README.md explicativo. Significa que cualquier auditor o profesor puede entender el propósito y los archivos de cada materia de forma independiente.